

Guardians : Pipeline d'Agent RAG Juridique

Module IA et Applications

Automatisation de la recherche juridique par agents IA

Code source : github.com/Lynchpin707/Guardians

Membres de l'équipe :

Lina Raoui	Mouhsine Elaoula
Soufiane Eloulji	Aymane Amraoui

Encadrante : Dr. Douae Ahmadoun

2026

Introduction générale

La recherche juridique constitue une tâche chronophage et exigeante : identifier les précédents pertinents dans des corpus de jurisprudence volumineux requiert une expertise pointue et de nombreuses heures de travail. L’essor des grands modèles de langage (LLM) et des architectures de type *Retrieval-Augmented Generation* (RAG) ouvre de nouvelles perspectives pour automatiser ce processus.

Guardians est un prototype (*Proof of Concept*) développé à l’ENSIAS qui démontre comment une équipe d’agents IA spécialisés peut, à partir du récit brut d’un client, extraire les faits juridiques pertinents, interroger une base de jurisprudence locale, et produire automatiquement un mémo stratégique. Le projet s’appuie sur des technologies open-source : le framework multi-agents **CrewAI**, la base vectorielle **ChromaDB**, et le modèle de langage léger **Bonsai 1.7B** servi localement via `llama.cpp`. Le code source est disponible sur GitHub : github.com/Lynchpin707/Guardians.

Ce rapport présente l’état de l’art et les choix technologiques retenus, l’architecture conçue, les détails d’implémentation, puis discute des résultats, limites et perspectives.

1 État de l’art et choix technologique

1.1 Retrieval-Augmented Generation (RAG)

Le paradigme RAG, introduit par Lewis *et al.* (2020), pallie les limitations des LLM purs en combinant une phase de *récupération* de documents pertinents (via une recherche vectorielle) et une phase de *génération* conditionnée par ces documents. Dans le domaine juridique, cette approche est particulièrement adaptée : elle permet d’ancrer les réponses du modèle dans des textes de loi et des décisions réelles, réduisant ainsi les *hallucinations*.

1.2 Agents IA et orchestration multi-agents

Les architectures multi-agents, popularisées par des frameworks tels que LangChain, AutoGen et **CrewAI**, permettent de décomposer une tâche complexe en sous-tâches confiées à des agents spécialisés qui collaborent séquentiellement ou en parallèle. CrewAI offre une abstraction simple (rôle, objectif, outils) et s’intègre naturellement avec des bases vectorielles.

1.3 Choix technologiques

Composant	Outil retenu	Justification
Orchestration agents	CrewAI	Simplicité, documentation riche
Base vectorielle	ChromaDB	Légèreté, embarquable, open-source
Modèle de langage	Bonsai 1.7B (llama.cpp)	Exécution locale, confidentialité
Données juridiques	Dataset HuggingFace	50 arrêts <i>Cour de cassation</i>
Langage	Python 3	Écosystème IA mature

2 Conception de l'architecture

2.1 Vue d'ensemble

Le pipeline est organisé en trois couches : **couche données** (ingestion et indexation dans ChromaDB), **couche agents** (trois agents CrewAI en séquence), et **couche sortie** (mémo juridique structuré).

2.2 Les trois agents

Agent 1 L'Extracteur Juridique

Reçoit le récit brut du client et extrait uniquement les faits objectifs et juridiquement pertinents (parties, actes, préjudices allégués), éliminant tout biais narratif.

Agent 2 L'Analyste RAG

Prend les faits normalisés et interroge ChromaDB par similarité sémantique pour récupérer les k arrêts les plus proches et en extraire les principes juridiques applicables.

Agent 3 Le Rédacteur Stratégique

Synthétise faits et jurisprudence pour rédiger un mémo professionnel : qualification juridique, probabilité de succès, risques et recommandations.

2.3 Flux de données

Récit client $\longrightarrow$ [Extracteur] $\longrightarrow$ [Analyste RAG $\leftrightarrow$ ChromaDB] $\longrightarrow$ [Rédacteur] $\longrightarrow$ Mémo juridique

3 Implémentation technique, résultats, limites et perspectives

3.1 Implémentation technique

L'ingestion (`ingest.py`) télécharge 50 arrêts de la *Cour de cassation* depuis le dataset `antoinejeannot/jurisprudence` (HuggingFace), génère les embeddings et les persiste dans `chroma_db/`. Bonsai 1.7B est servi via l'interface OpenAI-compatible de `llama.cpp` sur le port 8080. Pour prévenir le dépassement de la fenêtre de contexte, `rag_tools.py` tronque les documents récupérés à environ 600 caractères. CrewAI orchestre les trois agents séquentiellement dans `main.py`, la sortie de chaque agent alimentant le suivant.

3.2 Résultats Exemple d'exécution

Le pipeline a été testé sur le cas suivant : « *Mon employeur m'a licencié pour faute grave sans préavis parce que j'ai imprimé 3 pages de documents personnels sur l'imprimante du bureau.* »

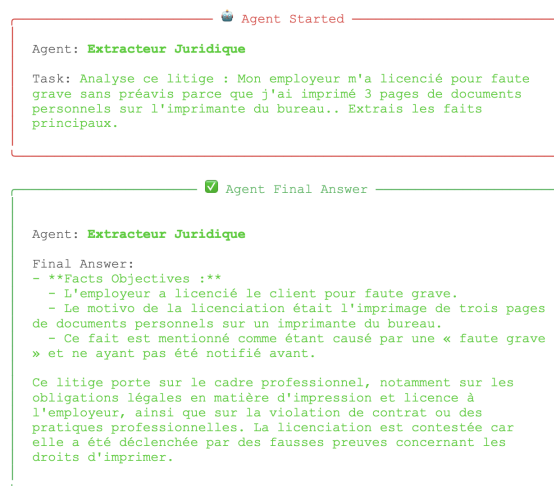


Figure 1: Extracteur Juridique

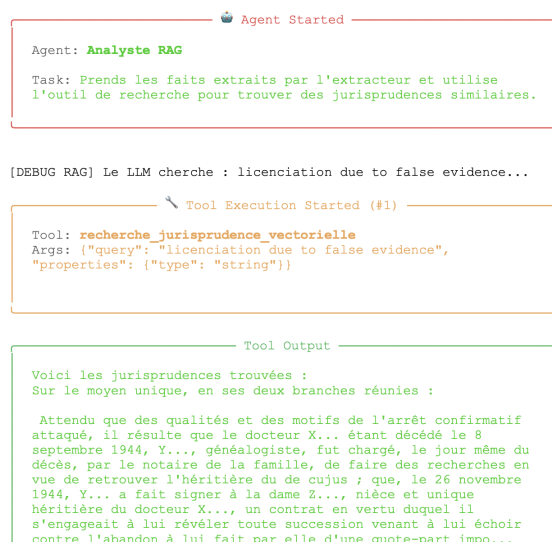


Figure 2: Analyste RAG

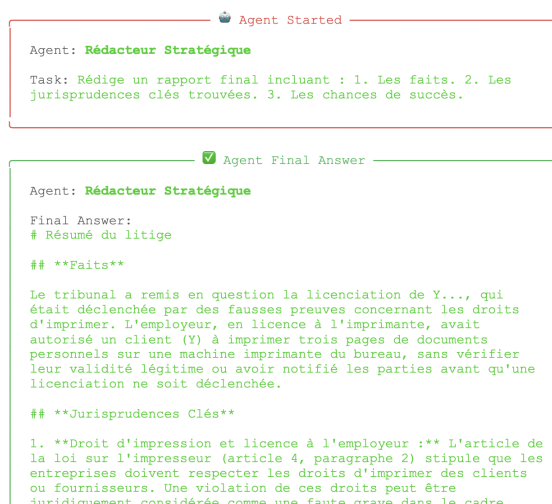


Figure 3: Rédacteur Stratégique

3.3 Limites identifiées

La troncature à 600 caractères perd parfois des éléments de motivation importants. Le modèle 1.7B manque de profondeur pour les nuances procédurales, et le corpus est limité à 50 arrêts, ce qui restreint fortement le rappel. Par ailleurs, la requête RAG a retourné des arrêts thématiquement éloignés (succession, assurances), symptôme d'un corpus trop restreint et insuffisamment spécialisé en droit du travail. Enfin, aucune évaluation formelle (métriques RAGAS, revue d'experts) n'a été conduite.

3.4 Perspectives

Plusieurs axes d'amélioration sont envisageables : augmenter le corpus à plusieurs milliers d'arrêts spécialisés ; remplacer Bonsai par un modèle plus grand (7B+) avec quantification GGUF ; implémenter un mécanisme de *re-ranking* ; et ajouter une interface web avec évaluation automatisée de la qualité des mémos.

Conclusion générale

Guardians démontre la faisabilité d'un assistant de recherche juridique entièrement local, fondé sur une architecture multi-agents RAG. Malgré les contraintes inhérentes à l'utilisation d'un modèle léger, le prototype valide le principe : décomposer la tâche complexe de conseil juridique en étapes spécialisées améliore significativement la qualité du résultat final.

Ce POC constitue une base solide pour un développement vers un outil opérationnel dans des cabinets d'avocats ou des services juridiques d'entreprises, où la confidentialité des données et l'explicabilité des décisions sont primordiales. Le code est librement accessible à l'adresse : github.com/Lynchpin707/Guardians.